Operating Systems and System Programming

Fall 2025

Chapter III: Memory Management

III.1. Overview

Memory management is an important responsibility of OS. It ensures that processes have access to memory when needed, prevents interference between processes, and optimizes system performance.

In this chapter, important concepts related to memory management are introduced and discussed.

Outline

- What is Memory Management?
- Memory Hierarchy
- Memory Allocation Techniques
- Conclusion
- Lab

Outline

- **What is Memory management?**
- Memory Hierarchy
- Memory Allocation Techniques
- Conclusion
- Lab

III.2. What is Memory Management?

Memory Management is the process of ***allocating, tracking,*** *and* ***freeing memory*** in a computer system to ensure ***efficient execution of program***.

The main goals of memory management include:

- **Allocation** : Provides memory to processes when needed.
- **Protection** : Prevents processes from accessing unauthorized memory.
- **Sharing** : Allows controlled memory sharing between processes.
- **Optimization** : Minimize fragmentation and maximize utilization.

Outline

- What is Memory Management?
- **Memory Hierarchy**
- Memory Allocation Techniques
- Conclusion
- Lab

III.3. Memory Hierarchy

The memory hierarchy in computer architecture refers to ***the structured arrangement of different types of memory based on their speed, cost,*** *and* ***capacity***.

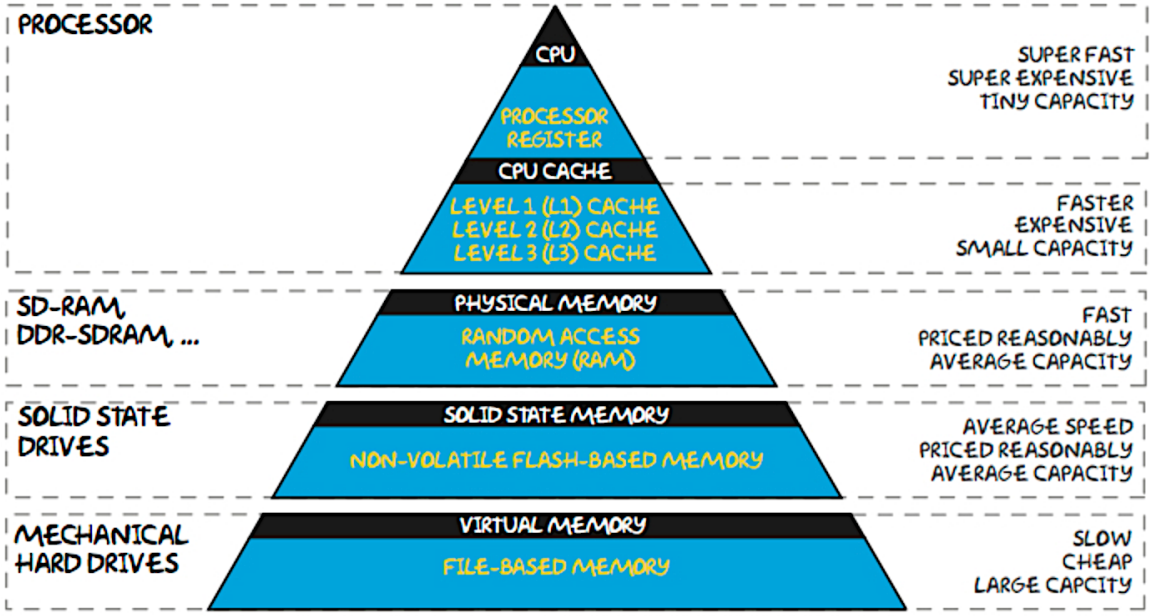
This design helps optimize system performance by ensuring that the most frequently accessed data is stored in the fastest memory available.

It is characterized by:

- **Response time**: Faster memory is placed **closer to the CPU to reduce access time**.
- **Cost vs. Capacity**: Faster memory (like cache) is more expensive and smaller, while slower memory (like hard drives) is cheaper and larger.

- **Locality of reference**: Programs tend to access the same data repeatedly (temporal locality) or data near recently accessed data (Spatial locality), which justifies storing it in faster memory.

Level of memory hierarchy can be represented as follows:



Source: https://zjustin.wordpress.com/2016/10/11/computer-memory-reflection/

Level of memory hierarchy can be represented as follows:

Level	Speed	Size	Cost per bit	Example
CPU Registers	Fastest	Very small	Very High	Temporary computation storage
Cache (L1, L2, L3)	Very Fast	Small	High	Frequently used instructions
Main Memory (RAM)	Moderate	Medium	Moderate	Active programs and data
Secondary Storage	Slow	Large	Low	HDDs, SSDs
Tertiary Storage	Slowest	Very large	Very Low	Backup tapes, optical disks
Virtual memory	Slowest	Very large	Lowest	Swap space on disk

Outline

- What is Memory management?
- Memory hierarchy
- **Memory Allocation Techniques**
- Conclusion
- Lab

III.4. Memory Allocation Techniques

They are fundamental to how operating systems manage and assign memory to processes.

They can be explained as:

1. **Contiguous Memory Allocation**: Processes are allocated memory in continuous blocks.

Thus, there are:

A. Fixed partitioning: Memory is divided into fixed-size partitions at boot time.

Characteristics	Problems
• Each partition can hold exactly one process • Partition sizes may be equal or unequal	• Internal fragmentation: Unused memory within a partition if the process is smaller than the partition • Limitation: Large processes may not fit if partitions are too small.

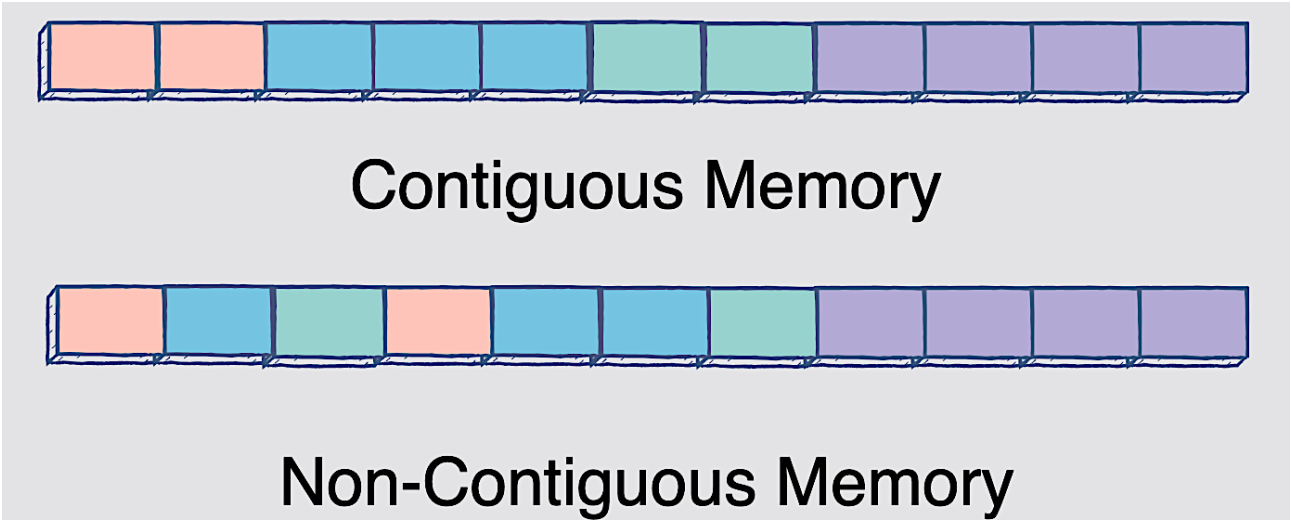
B. Dynamic partitioning: Memory is allocated as needed in variable-sized blocks.

Characteristics	Problems
• Each process gets exactly the memory it requires • More flexible than fixed partitioning	• External Fragmentation: Small gaps between allocated blocks that cannot be used. • Compaction: required periodically to consolidate free space

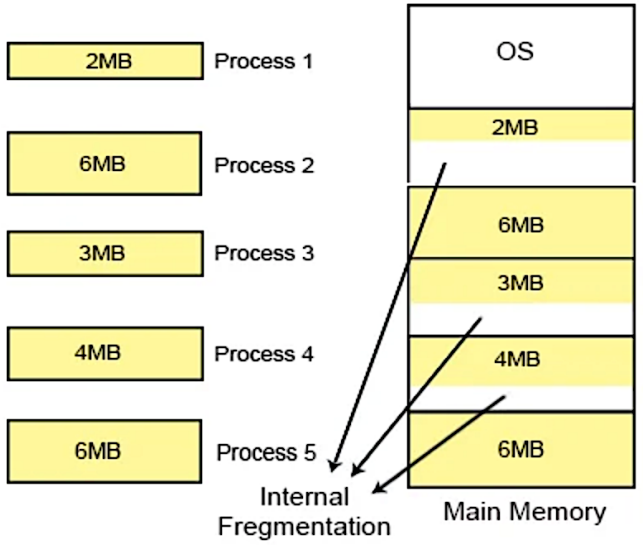
2. Non-Contiguous Memory Allocation:

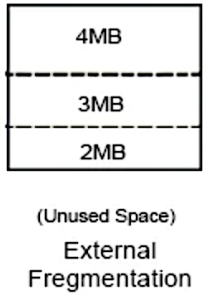
Processes are allocated memory in non-adjacent blocks.

Thus, We have:

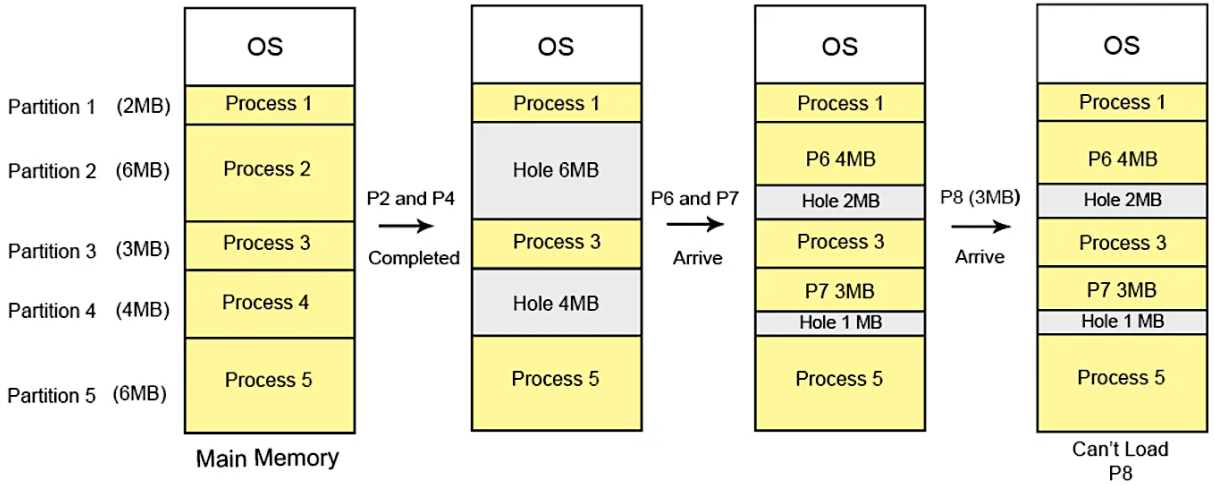

Contiguous Memory
Non-Contiguous Memory

Fixed Partitioning



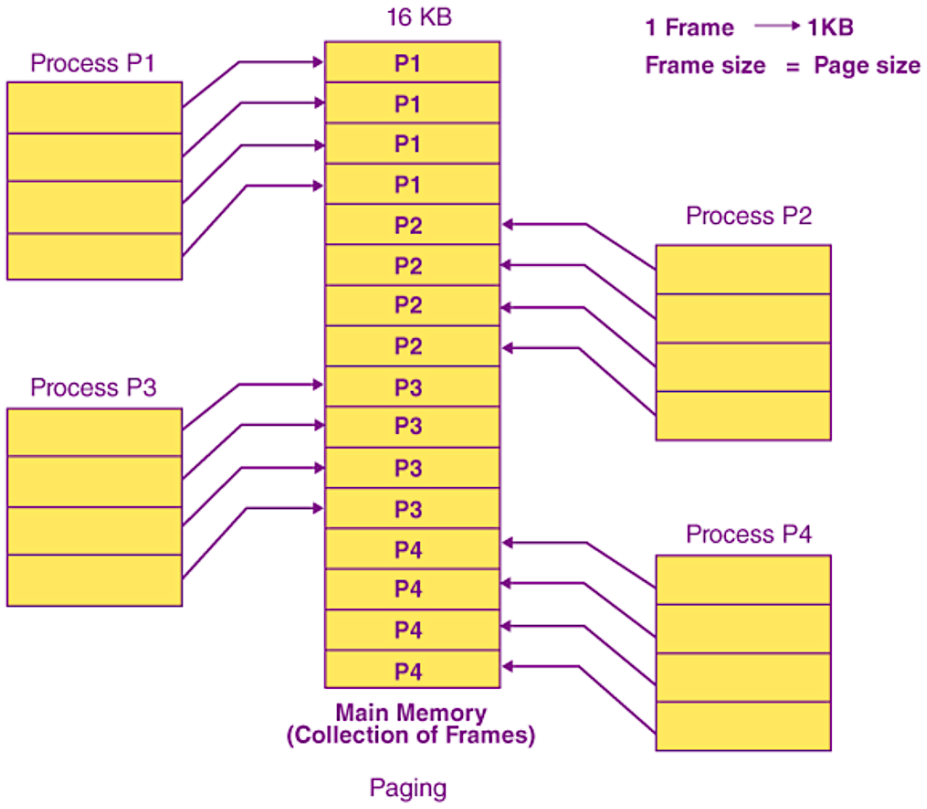


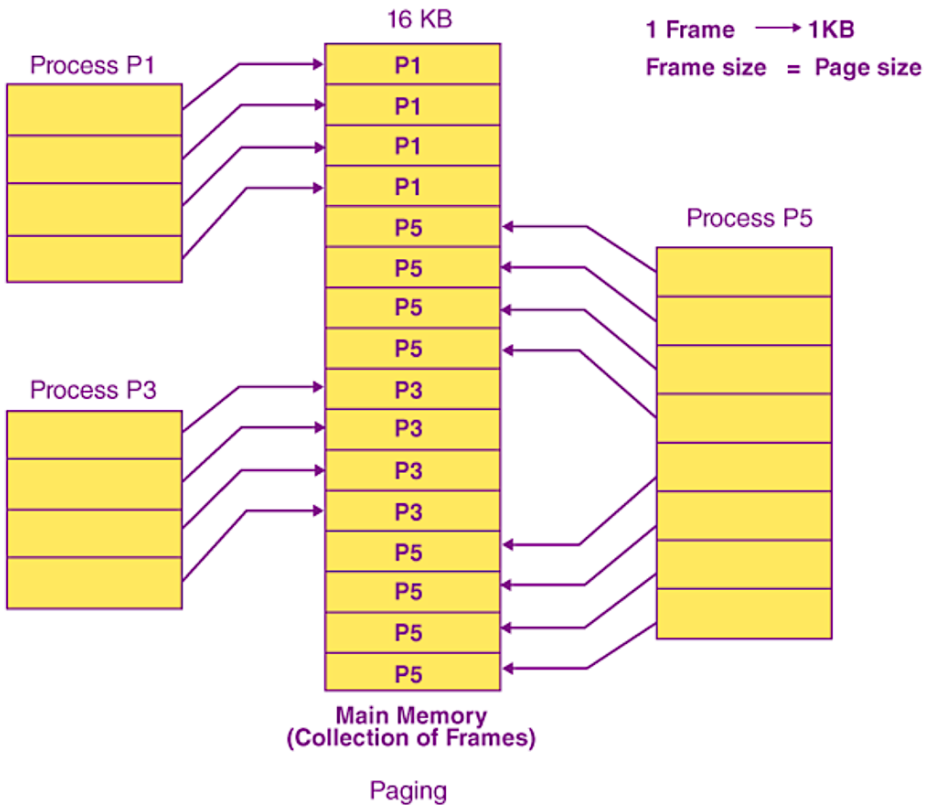
Dynamic Partitioning



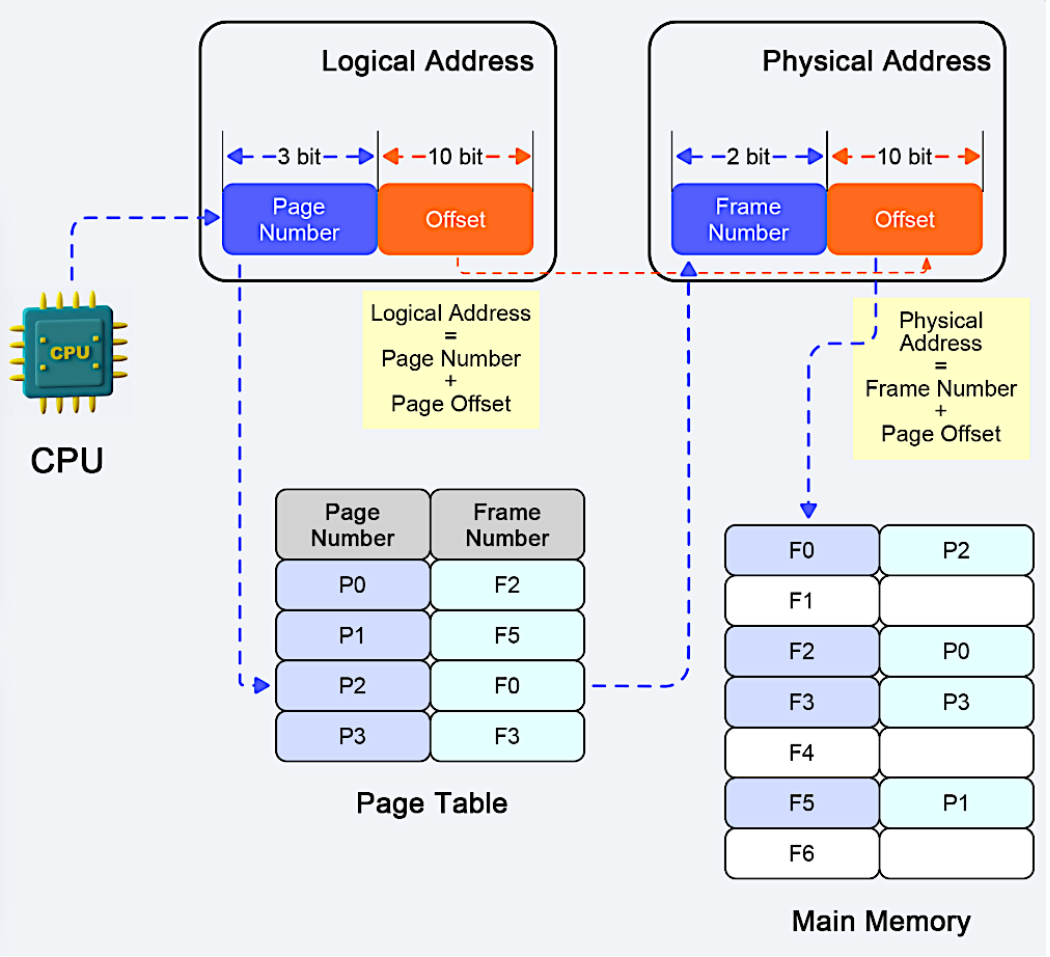
Type	Characteristics	Advantages	Disadvantages
Paging	• Divides memory into fixed-size blocks called frames • Divides processes into same-sized blocks.	• No external fragmentation • Allows virtual memory (pages can be swapped to disk)	• Small internal fragmentation possible (last page may not be full) • Overhead of page table management
Segmentation	• Divides memory into variable-sized segments matching program structure (code, data, stack) • Uses a segment table with base addresses and lengths.	• Matches programmer's view of memory. • Easy sharing of segments (e.g., shared libraries)	• External fragmentation can occur. • Complex memory management
Segmented paging (Hybrid)	• Combines segmentation and paging. • Segments are divided into pages. • Used in modern systems like x86 architecture.		

Paging

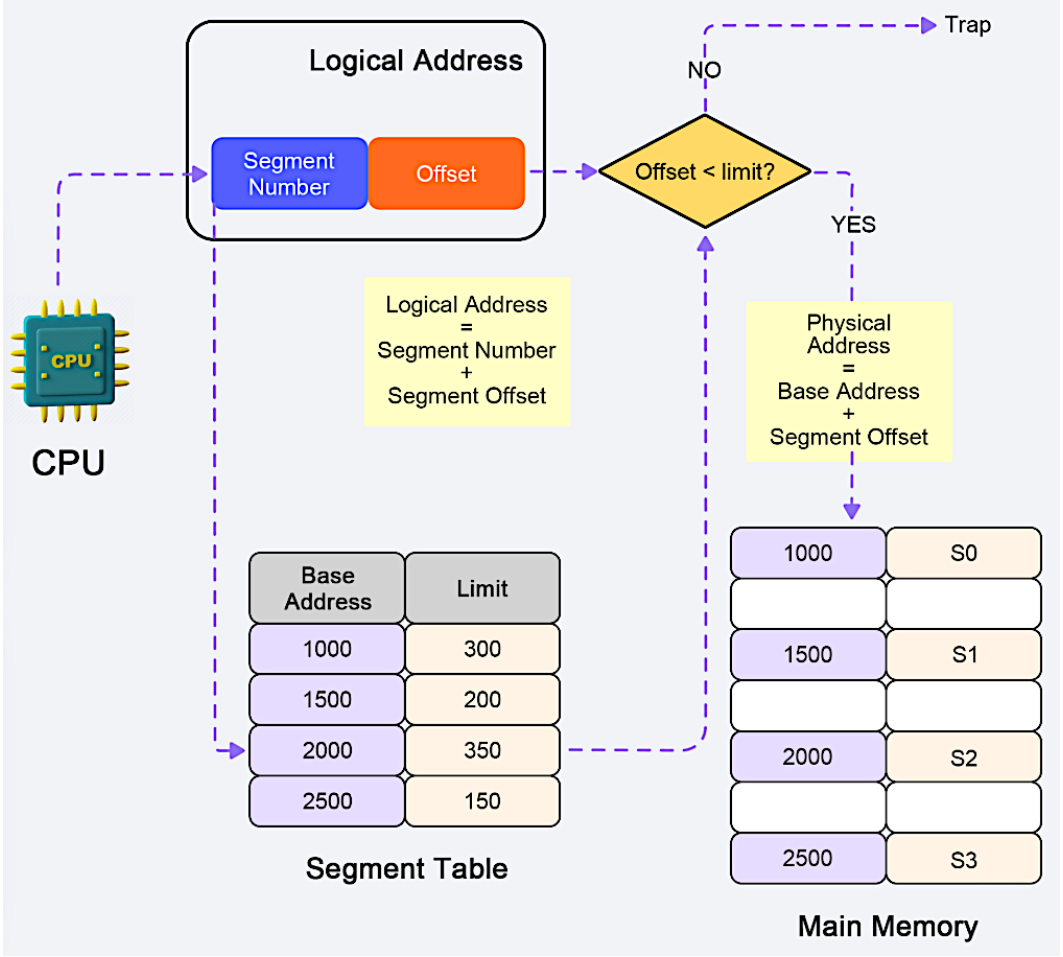

16 KB
1 Frame → 1KB
Frame size = Page size
Process P1
Process P2
Process P3
Process P4
P1
P2
P3
P4
Main Memory
(Collection of Frames)
Paging


16 KB
1 Frame → 1KB
Frame size = Page size
Process P1
Process P5
Process P3
P1
P5
P3
Main Memory
(Collection of Frames)
Paging

Paging



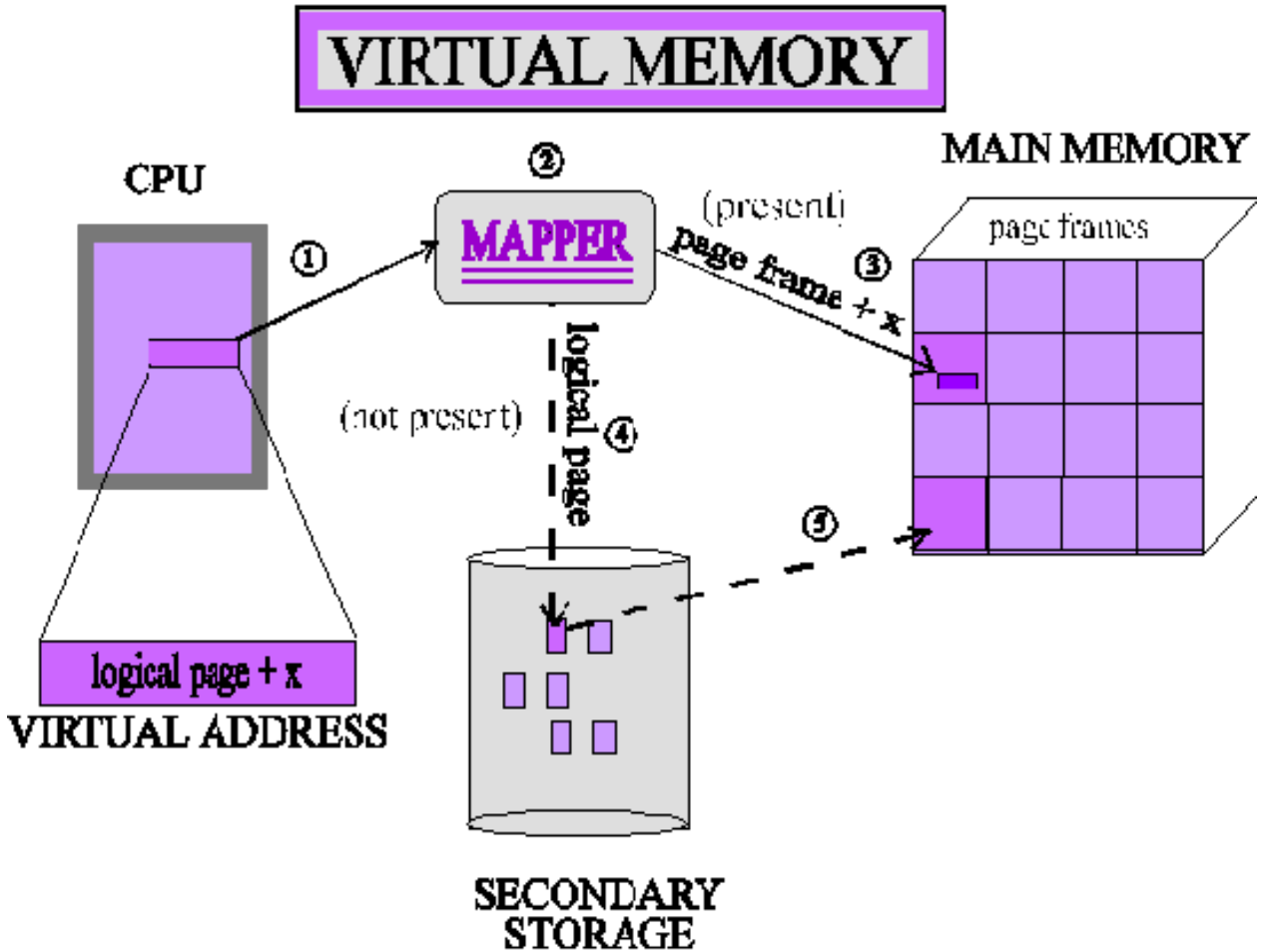
Segmentation



Source: https://bytebytego.com/guides/what-are-the-differences-between-paging-and-segmentation/

3. Virtual Memory Techniques:

It is a mechanism used to manage memory using **hardware** and **software**. It is **a part of the secondary storage** that gives the user the illusion that it is a part of the main memory. It helps in running multiple applications with low main memory and increases the degree of multiprogramming in systems. It is commonly implemented using demand paging.



Source: https://terpconnect.umd.edu/~ebyrd/proj/virtual.html

Techniques	Characteristics
Demand paging	Pages are loaded into memory only when needed. o Uses page faults to trigger loading from disk.
Page replacement algorithms	Used when free frames are unavailable, through: FIFO (First-In-First-Out) o Replaces the oldest page. o Simple but poor performance. LRU (Least Recently Used) o Replaces the least recently accessed page. o Near-optimal but expensive to implement Optimal Algorithm o Replaces the page not needed for longest time.

4. Buddy System Allocation:

Divides memory into power-of-2 sized blocks.

- Process:
  - If a request is made, the system finds the smallest sufficient block
  - If not available, splits a larger block into two "buddies"
  - Merges adjacent free buddies when memory is freed
- Used in Linux kernel for efficient allocation

5. Slab Allocation:

Allocates memory for kernel objects. Its pre-allocates "slabs" of memory for frequently used objects.

Advantages:

- No fragmentation for small objects
- Fast allocation/deallocation

Comparison of Techniques

Technique	Contiguous?	Fragmentation	Overhead	Used For
Fixed Partitioning	Yes	Internal	Low	Early OS
Dynamic Partitioning	Yes	External	Medium	-
Paging	Yes / No	Small internal	Medium	Modern OS
Segmentation	Yes / No	External	High	Program memory
Buddy System	Yes/No	Minimal	Medium	Kernel memory
Slab Allocation	No	None	Low	Kernel objects

Outline

- What is Memory Management?
- Memory Hierarchy
- Memory Allocation Techniques
- **Conclusion**
- Lab

III.5. Conclusion

Memory management is essential for system stability and performance.

This chapter introduced the memory hierarchy, explained paging, segmentation, and different existing memory management techniques applied in various cases.

This knowledge provides a broad understanding on the importance of memory management, as well as applied technique to manage them.

Outline

- What is Memory management?
- Memory hierarchy
- Memory allocation techniques
- Conclusion
- **Lab**

Practice: Simulating paging and Memory leak detection

1. Paging

Paging divides memory into fixed-size blocks (pages). It allows non-contiguous memory allocation and avoids external fragmentation.

Simulation idea:
- Use a list to represent physical memory.
- Map logical pages to physical frames.

2. Memory Leak Detector Using tracemalloc

Memory leaks can degrade performance or crash applications. tracemalloc helps pinpoint the source of leaks by tracking memory allocations over time.

Detect and analyze memory leaks in Python. programs using the built-in tracemalloc module.

1. Paging

import random

```
# Physical memory: 8 frames (some already occupied)
physical_memory = [None, 'OS', None, 'AppX', None, None, 'OS', None]

# Our program's logical pages
program_pages = ['A', 'B', 'C', 'D']  # Program thinks it has pages 0,1,2,3
# Page table: maps logical pages → physical frame
spage_table = {}
available_frames = [i for i, frame in enumerate(physical_memory) if frame is None]
print("Initial Memory:", physical_memory)
print("Available frames:", available_frames)

# Simulate loading program pages into available physical frames
for logical_page_num, page_content in enumerate(program_pages):
    if available_frames:
        # Randomly choose an available physical frame
        physical_frame_num = random.choice(available_frames)
        available_frames.remove(physical_frame_num)
        # Update page table: "When program asks for page X, give it frame Y”
        page_table[logical_page_num] = physical_frame_num
        # Load the page into physical memory
        physical_memory[physical_frame_num] = page_content
```

```
print("\nAfter loading program:")
print("Page Table (logical → physical):", page_table)
print("Physical Memory:", physical_memory)

# Simulate memory access: Program asks for "page 0 or1, 2, 3”
logical_address = 2 [← change values here]
physical_frame = page_table[logical_address]
print(f"\nProgram asks for page {logical_address}")
print(f"Page table says: go to frame {physical_frame}")
print(f"Content found: '{physical_memory[physical_frame]}'")
```

Output

Initial Memory: [None, 'OS', None, 'AppX', None, None, 'OS', None]
Available frames: [0, 2, 4, 5, 7]

After loading program:
Page Table (logical → physical): {0: 5, 1: 0, 2: 2, 3: 4}
Physical Memory: ['B', 'OS', 'C', 'AppX', 'D', 'A', 'OS', None]

Program asks for page 1
Page table says: go to frame 0
Content found: 'B'

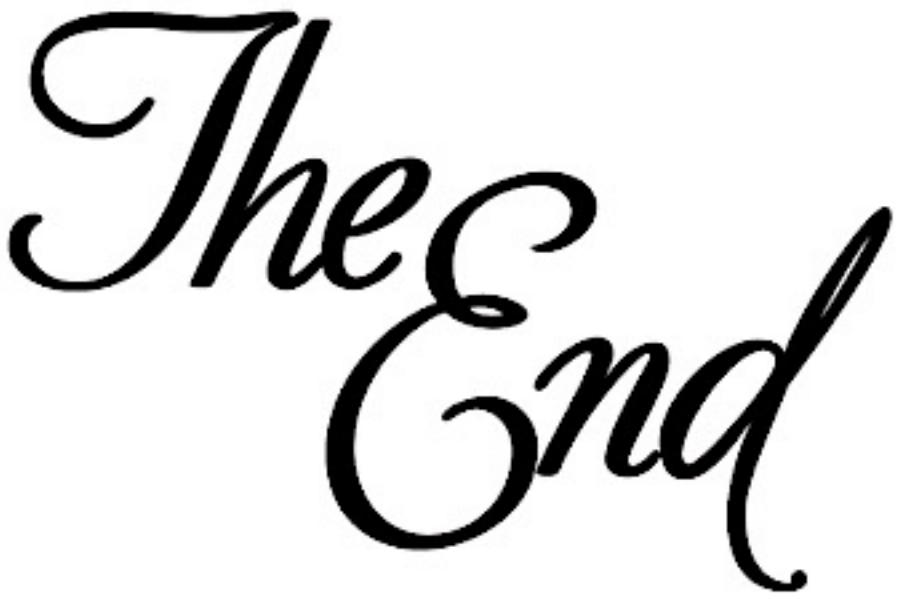
2. Memory leak detection

import tracemalloc

```
tracemalloc.start()
# Before
snap1 = tracemalloc.take_snapshot()
# Suspect code
data = ["leak" * 100 for _ in range(500)]
# After
snap2 = tracemalloc.take_snapshot()
# Detect leak
for stat in snap2.compare_to(snap1, 'lineno')[:3]:
    print(f"Potential leak: {stat}")
tracemalloc.stop()
```

Output

Potential leak: /var/folders/vy/dy757wz17p9bggh3h7vw044m0000gn/T/ipykernel_41190/475139897.py:9: size=4160 B (+4160 B), count=1 (+1), average=4160 B
Potential leak: /Users/newowner/Desktop/anaconda3/lib/python3.12/codeop.py:126: size=855 B (+569 B), count=5 (+3), average=171 B
Potential leak: /Users/newowner/Desktop/anaconda3/lib/python3.12/tracemalloc.py:560: size=312 B (+312 B), count=2 (+2), average=156 B


The End